

Documentação da Simulação — Somador de Ponto Flutuante 13 bits

Disciplina MCTA024 — Sistemas Digitais (UFABC) · Projeto: Somador de Ponto Flutuante em FPGA (DE10-Lite)

Este documento registra a simulação do circuito **fp_adder** feita para entender exatamente o que o código faz, o que acontece fisicamente na placa DE10-Lite ao mexer nas chaves, a faixa de valores que o circuito consegue representar e somar, casos de teste concretos prontos para gravar na placa, e a verificação de que o código do repositório do grupo é equivalente ao código original do livro-texto.

1. O formato de 13 bits

Cada número (operando) é representado com 13 bits divididos em três campos:

Campo	Tamanho	O que representa
sign	1 bit	1 = número negativo, 0 = número positivo
exp	4 bits	expoente, sem sinal (0 a 15)
frac	8 bits	fração/significando, sem sinal, lido como 0,xxxxxxx em binário

Fórmula do valor representado

$$\text{valor} = (-1)^{\text{sinal}} \times 0,\text{fração} \times 2^{\text{expoente}}$$

Para o número estar em formato válido (“normalizado”), o bit mais significativo da fração precisa ser 1 — exceto quando o número é zero.

2. Faixa teórica do formato (13 bits livres)

Se fosse possível escolher os 13 bits livremente (sem passar pelas chaves físicas da placa), esta é a faixa de magnitude que o formato consegue representar:

Limite	Configuração de bits	Valor
Menor número normalizado (não-zero)	frac = 10000000, exp = 0000	0,5
Maior número possível	frac = 11111111, exp = 1111	32.640

Essa é a faixa *teórica* do formato. A faixa que dá pra testar de fato na placa física, apertando chaves, é bem menor — ver seção 4.

3. Como as chaves da DE10-Lite viram os dois números

O código do somador (fp_adder.vhd) não sabe nada sobre chaves ou displays — ele só recebe dois números de 13 bits e devolve um número de 13 bits somado. Quem faz essa tradução é o arquivo de topo fp_adder_de10lite.vhd, que conecta as 10 chaves (SW) e os 2 botões (KEY) da placa aos bits de entrada.

Número 1 — quase todo fixo no código

sign1 é sempre 0 (positivo) e exp1 é sempre “1000” (=8), ambos fixos no código. Só a fração usa 2 chaves de verdade: frac1 = '1' & SW(1) & SW(0) & “10101” — o bit mais significativo é fixo em 1 (já vem normalizado) e o final “10101” também é fixo. Isso deixa o número 1 restrito a apenas 4 valores possíveis.

SW1	SW0	frac1 (binário)	Valor do número 1
0	0	10010101	149
0	1	10110101	181
1	0	11010101	213
1	1	11110101	245

Número 2 — o que você realmente controla

sign2 = SW(9) (sinal), frac2 = '1' & SW(8 downto 2) (7 chaves livres + bit fixo no topo), exp2 = “10” & KEY(1) & KEY(0) (os 2 botões formam os 2 bits menos significativos do expoente; os 2 mais significativos são fixos em “10”, ou seja, exp2 só varia entre 8 e 11).

Por que fixar quase tudo do número 1?

A placa só tem 10 chaves + 2 botões = 12 entradas físicas, e o somador precisa de 26 bits (13 + 13) para os dois operandos. Não cabe ter os dois totalmente livres — por isso um deles foi fixado quase por completo no código, sobrando as poucas entradas físicas para o outro. Na prática, ao mexer nas chaves, quem você está de fato escolhendo é o número 2 — o número 1 muda muito pouco (só entre 149 e 245).

Saídas — sem multiplexação de tempo

Diferente da placa do livro (que multiplexava um único display no tempo), a DE10-Lite tem displays dedicados: exp_out vai direto para o HEX2; frac_out(7:4) vai para o HEX1; frac_out(3:0) vai para o HEX0; sign_out acende o LEDR(9).

4. Faixa de valores realmente alcançável na placa física

Operando	Faixa alcançável	Observação
Número 1	149, 181, 213 ou 245	sempre positivo, só 4 valores possíveis
Número 2	de ± 128 até ± 2040	passo de 1 (perto de 128) até passo de 8 (perto de 2040), conforme KEY
Soma máxima alcançável		$245 + 2040 = 2285$
Soma mínima alcançável		$149 - 2040 = -1891$

Note que 2285 é bem menor que os ≈ 32.640 que o formato de 13 bits aguentaria em teoria (seção 2). A limitação vem inteiramente do mapeamento de chaves escolhido, não do formato em si.

5. Exemplo pedido: 1500 + 4000

Por que esse caso não roda nas chaves

4000 está fora do alcance físico (número 2 não passa de 2040) e 1500 não é um dos 4 valores possíveis do número 1. Esse par de valores **não dá para ser montado apertando chaves na placa** — só é testável via simulação (testbench/GHDL), que não tem essa limitação.

Como ficaria via simulação (testbench)

Operando	Bits (sign / exp / frac)	Valor representado
Número 1 (alvo: 1500)	0 / 1011 / 10111100	1.504,000 (aproximado — ver nota abaixo)
Número 2 (alvo: 4000)	0 / 1100 / 11111010	4.000,000 (exato)
Resultado da soma	0 / 1101 / 10101100	5.504,000

Nota sobre a imprecisão

1500 não é representável de forma exata nesse formato, porque a mantissa (fração) só tem 8 bits. O valor mais próximo que o formato consegue guardar é 1504. Por isso a soma dá 5504 em vez do “5500 matemático” — não é um bug do circuito, é uma limitação do formato de ponto flutuante simplificado (perda de precisão / truncamento), e vale citar isso no relatório.

6. Casos de teste prontos para a placa física

Configurações de chave calculadas para cobrir os principais comportamentos do circuito, com o resultado exato esperado em cada display.

Caso	SW 9	SW8..2	KEY1 KEY0	SW1 SW0	Número 1	Número 2	Soma	HEX2 HEX1 HEX0	LEDR9
1 — soma simples	0	0000000	0 0	0 0	+149	+128	+276	9 8 A	apagado
2 — carry-out	0	1111111	1 1	1 1	+245	+2040	+2272	C 8 E	apagado
3 — subtração c/ deslocamento	1	0010110	0 0	0 0	+149	−150	−1	1 8 0	ACESO
4 — resultado negativo	1	1111111	1 0	0 0	+149	−1020	−872	A D A	ACESO
5 — underflow tentado*	1	0000000	0 0	0 0	+149	−128	+21	5 A 8	apagado

*Nota sobre o Caso 5

O Caso 5 é proposital: mostra por que o resultado “virar zero” (o terceiro caso especial de normalização, para quando o resultado é pequeno demais) **nunca acontece apertando chaves** — porque o expoente do número 1 nunca é menor que 8, e o deslocador de normalização só conta até 7 zeros à esquerda. Esse caso só é demonstrável via testbench, não fisicamente na placa — vale citar isso no relatório como uma limitação consciente do mapeamento escolhido.

7. Comparação: código do livro vs. código do repositório

Resultado da comparação

VALIDADO — o código de `fp_adder.vhd` do repositório (`rtl_original/fp_adder.vhd`) é estruturalmente idêntico ao Listing 3.19 do livro-texto (Pong P. Chu, *FPGA Prototyping by VHDL Examples*, seção 3.7.4).

A comparação foi feita removendo espaços, comentários e diferenças de formatação de ambos os arquivos e comparando o texto resultante. As únicas diferenças encontradas foram puramente cosméticas:

Diferença encontrada	Impacto
Livro usa '0' (aspas simples); repositório usa "0" (aspas duplas) em um trecho de concatenação	Nenhum — ambas as formas são válidas em VHDL nesse contexto
Livro tem parênteses extras em algumas condições, ex: <code>(sum(7)=1)</code> ; repositório não tem	Nenhum — apenas estilo de escrita, mesma lógica booleana
Comentários do repositório estão traduzidos para português	Nenhum — comentários não afetam a compilação

Entidade, portas, sinais internos e os quatro estágios (sort, align, add/sub, normalize) são idênticos em nome, tipo e ordem. Não foram encontradas diferenças de lógica. A Etapa 1 do projeto (validar o VHDL original) pode ser considerada correta em relação ao material de apoio.

Documento gerado a partir de simulação em Python (modelo golden bit-exato do VHDL) e da comparação textual entre o código do repositório e o material de apoio da disciplina. Os valores desta simulação complementam — e não substituem — a simulação oficial em GHDL/GTKWave exigida pelo enunciado do projeto.